

UNLOCKING THE BENEFITS OF DATABASE DEVOPS AND AUTOMATION



Kellyn Gorman
Multiplatform Advocate and Engineer
Redgate

Thank you to ALL of our sponsors! - Be sure to stop by all tables!

Platinum



School of Computing
College of Computing,
Engineering and Construction



redgate +



Simple Talk

Silver



Gold



SOLARWINDS®



COZYROC



SQLGrease



ENTERPRISE
INTEGRATION

onecall®

In-Kind

O'REILLY®





Monthly Meetings
3rd Wednesday of each month
jssug.org

SQL Clinic

**Get your SQL questions answered here
by our SQL experts during our breaks!**

(Across from Registration)

#SQLSatJax

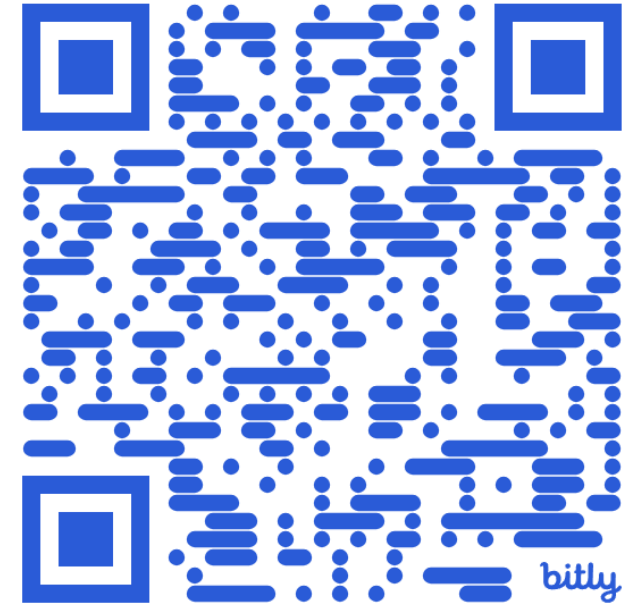


501 Legion Charitable Donation For Housing

Thank the 501 Legion for Supporting Our Event!

JSSUG Will Match Donations up to \$500

Donation Bucket on Registration Table



Our vision for Ft. Barnabas is to one day be a bastion of hope and stability in guiding veterans on their path to stability. Fort Barnabas will be constructed as a mixture of portable tiny homes, or “Barnabas Bungalows,” and brick-and-mortar condo-style family homes. <https://operationbarnabas.com/get-involved/>

Session Evaluations

Your feedback is important to us!

Please fill out and hand to speaker after the session!

Kellyn Gorman

Engineer and Advocate



Kellyn.Gorman@red-gate.com



<https://linkedin.com/in/kellyngorman>



<https://dbakevlar.com>



Agenda

- Why Database DevOps Matters
- Database DevOps Tools
- Stepping Through an Example Deployment
- Scripting Example
- Recommended Practices
- Summary



WHY DATABASE DEVOPS MATTERS





DevOps for data matters because it brings the same principles of automation, collaboration, and continuous integration/continuous delivery (CI/CD) that have transformed application development to the world of databases and data management.

ENSURES DATA INTEGRITY AND SECURITY

- Traditional database changes are often manual, increasing the risk of human errors.
- DevOps practices enforce version control, automated testing, and validation before deploying schema changes.
- Helps maintain compliance with regulatory standards (GDPR, HIPAA, etc.).

ACCELERATES DEVELOPMENT AND DEPLOYMENT

- Automating database migrations allows development teams to release new features faster without waiting for manual DB updates.
- Reduces bottlenecks in database change approval processes.

MINIMIZES DOWNTIMES AND FAILURES

- Infrastructure-as-Code (IaC) and automated rollback mechanisms ensure stability.
- Introduces Data-as-Code(DaC), which is a concept around baselining data before a release, ensuring not just code changes are recoverable, but data consistency as well.
- Database migrations are tested in lower environments before reaching production, reducing failed deployments.

ENHANCES COLLABORATION

Between Developers and DBAs



Encourages shared responsibility for database changes rather than treating DBAs as gatekeepers, which we've been viewed as for decades.



Version-controlled migrations in repositories (e.g., Flyway, Liquibase) provide transparency.

IMPROVES DATA QUALITY AND ANALYTICS

- With automated testing and monitoring, data consistency across environments is enforced.
- Ensures analytics and reporting teams work with the most up-to-date and accurate data.

ENABLES SCALABILITY

1

For Cloud and Hybrid Environments

2

Cloud-native databases require rapid provisioning, replication, and backup automation.

3

DevOps practices ensure databases scale dynamically to meet demand.

**BY TREATING DATA AS PART OF THE
DEVOPS LIFECYCLE, ORGANIZATIONS
GAIN SPEED, RELIABILITY, AND
COMPLIANCE WHILE REDUCING RISK.**

+

•

o

DATABASE DEVOPS TOOLS



DATABASE VERSION CONTROL AND SCHEMA MANAGEMENT



Flyway – Version control for database migrations, supports multiple databases.



Liquibase – Schema change automation with rollback support.



Delphix/Test Data Mgr Suite – To mask and subset data to non-prod environments.

CI/CD Pipelines for Databases

GitHub Actions / GitLab CI/CD – Automates database migrations alongside application code.

Azure DevOps – Integrates with SQL Server, PostgreSQL, and other databases.

Jenkins – Automates database deployment and rollback tasks.



INFRASTRUCTURE AS CODE(IAC)



Terraform – Manages database infrastructure across cloud environments.



Ansible – Automates database provisioning and configuration.



Pulumi – Defines database infrastructure using modern programming languages.

DATA-AS-CODE

Use a transaction with Try/CATCH with DDL or use tools that can rollback changes with UNDO scripts.

A light blue downward-pointing arrow indicating a flow from the first step to the second.

Azure Data Factory, Golden Gate and other products allow data baseline files that can be checked into repositories, just as easy as code.

A light blue downward-pointing arrow indicating a flow from the second step to the third.

Storage snapshots offer small and simple snapshots in time of data to use for rollback purposes.

TRY/CATCH METHOD

```
BEGIN TRY
    BEGIN TRANSACTION;

    -- Risky DDL
    ALTER TABLE Customers DROP COLUMN Column6;

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
END CATCH;
```

THE PROBLEM WITH TRY/CATCH METHOD

- This is error handling, not ROLLBACK.
- Should have exit criteria and allowable errors built in.
- Should consider dependencies.
- Ensure that failures are trapped and alerted upon.

CREATE A BEFORE IMAGE/SCRIPTS

Before running a DDL change:

- Script out the current table, view, procedure, etc.
- Save it in source control or a .sql file.

Then if something breaks, you can manually **revert** the change by running the saved script.

Best for: Controlled environments, versioning, and DevOps pipelines.

USE SOURCE CONTROL/BASELINE

Tools like **Redgate SQL Compare**, **dbForge**, **Flyway**, or **Visual Studio SSDT** track schema changes.

You can:

- Compare current DB to a known good version
- Generate a rollback script
- Apply it safely

Best for: CI/CD pipelines and team-based development.

CREATE SNAPSHOTS BEFORE RELEASES

```
CREATE DATABASE MyDB_Snapshot ON  
(  
    NAME = MyDB_Data,  
    FILENAME = 'C:\Snapshots\MyDB_Snapshot.ss'  
)  
AS SNAPSHOT OF MyDB;
```

Best for: Critical Issues, as it restores the entire database back to that point in time.

PREPARE FOR THE FOLLOWING

- DROP TABLE, DROP PROCEDURE, etc.
- ALTER TABLE that drops columns
- TRUNCATE TABLE (for DML, but still irreversible)

Backup

Copy

Verify

Database Testing and Validation

pgTAP (PostgreSQL) – Unit testing framework for database objects.

tSQLt (SQL Server) – Test framework for stored procedures and views.

DbFit – Database integration testing within CI/CD pipelines.



PERFORMANCE MONITORING AND OBSERVABILITY



Prometheus + Grafana – Real-time database monitoring and alerting.



New Relic – Monitors query performance, slow-running transactions.

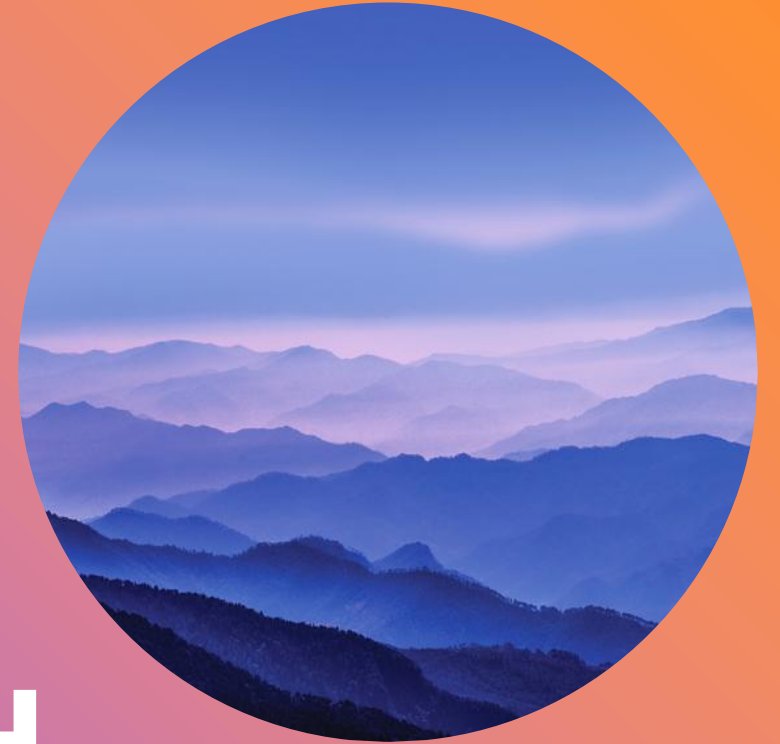


Datadog – Tracks database health and resource utilization.



Redgate Monitor – Multiplatform health and observability

STEPPING THROUGH AN EXAMPLE DEPLOYMENT



STORY OF A DEPLOYMENT



- A single pane of glass orchestrator
- Identifying the changes to be performed.
- Setup of the environment, including configurations and prechecks
- Deployment of changes, logging as steps are performed.
- “Error scaling” to exit in case of critical errors and undo scripts presented.
- If no errors, success criteria and log releases.
- Changes documented and drift cleared.

+

•

○

SCRIPTING EXAMPLE

Standard Data DevOps

In our DevOps example, we'll:

- Ensure you're in a **clean** Git state.
- **Pull** latest changes.
- Run Flyway **migration**.
- Confirm or test **undo script(s)**.
- Optionally **tag** the release in Git.





Step 1:

SET UP OUR PROJECT

USING FLYWAY AS THE ORCHESTRATOR

```
FlywayProject/  
├─ sql/  
│   ├── V1__Transform_Customers_With_Extension.sql  
│   └─ U1__Undo_Transform_Customers_With_Extension.sql  
├─ flyway.conf  
└─ deploy.ps1
```

SETUP FLYWAY.CONF

```
flyway.url=jdbc:sqlserver://localhost:1433;databaseName=MyDatabase
flyway.user=MyUsername
flyway.password=MyPassword
flyway.locations=filesystem:sql
flyway.driver=com.microsoft.sqlserver.jdbc.SQLServerDriver
flyway.baselineOnMigrate=true
flyway.cleanDisabled=false
```



Step 2:

ACTUAL CHANGE SCRIPTS

BACKUP TABLE AND CREATE NEW TABLE

```
-- Step 1: Backup original Customers table with all data
SELECT * INTO Customers_backup FROM Customers;

-- Step 2: Create extended table with modified schema
-- We'll exclude the Email column and add two new ones
CREATE TABLE Customer_EXT (
    CustomerID INT PRIMARY KEY,
    Name NVARCHAR(100),
    PhoneNumber NVARCHAR(20) NULL,
    CreatedDate DATETIME,
    ModifiedDate DATETIME DEFAULT GETDATE()
);
```

V1__Transform_Customers_With_Extension.sql

INSERT DATA AND CREATE INDEX

-- Step 3: Insert existing data from Customers into Customer_EXT

-- Email is dropped; PhoneNumber and ModifiedDate are added

```
INSERT INTO Customer_EXT (CustomerID, Name, CreatedDate)
```

```
SELECT CustomerID, Name, CreatedDate
```

```
FROM Customers;
```

-- Step 4: Create a new index on Name for performance

```
CREATE NONCLUSTERED INDEX IX_Customer_EXT_Name ON Customer_EXT(Name);
```

UNDO CHANGES

```
-- Drop the index first (optional: IF EXISTS for safety)
IF EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Customer_EXT_Name')
    DROP INDEX IX_Customer_EXT_Name ON Customer_EXT;

-- Drop the Customer_EXT table
DROP TABLE IF EXISTS Customer_EXT;

-- Drop the backup table
DROP TABLE IF EXISTS Customers_backup;
```

+



Step 3:

AUTOMATION

POWERSHELL- DEFINE VARIABLES

```
# Deploy.ps1 - Flyway SQL Server deployment with undo and Git tagging

$flyway = "flyway"
$config = "flyway.conf"
$projectRoot = $PSScriptRoot
$sqlDir = Join-Path $projectRoot "sql"
$releaseTag = "release-" + (Get-Date -Format "yyyyMMdd-HH:mm:ss")

Push-Location $projectRoot
```

deploy.ps1

CLEAN THE REPOSITORY AND PULL

```
# Git status check
Write-Host "Checking Git status..."
$gitStatus = git status --porcelain
if ($gitStatus) {
    Write-Warning "Git working directory is not clean. Please commit or stash changes first."
    exit 1
}

# Pull latest
git pull
```

EXECUTE DEPLOYMENT

```
# Run migration
Write-Host "Running Flyway migration..."
& $flyway migrate -configFiles=$config
if ($LASTEXITCODE -ne 0) {
    Write-Error "Migration failed. Aborting deployment."
    Pop-Location
    exit 1
}
```

+



Step 4:

IF FAILURE

REPORT SUCCESSFUL OR CHOOSE UNDO

```
Write-Host "Migration completed successfully."

# Optional undo test
$undoConfirm = Read-Host "Would you like to test the undo script now? (y/n)"
if ($undoConfirm -eq "y") {
    Write-Host "Running Flyway undo..."
    & $flyway undo -configFiles=$config
    if ($LASTEXITCODE -ne 0) {
        Write-Error "Undo failed."
        Pop-Location
        exit 1
    }
    Write-Host "Undo executed successfully."
}
```

TAG RELEASE

```
# Git tag and push  
git tag $releaseTag  
git push origin $releaseTag
```

```
Write-Host "Release tagged as '$releaseTag'. Deployment complete."
```

```
Pop-Location
```

+

•

○

RECOMMENDED PRACTICES

TREAT DATABASE CHANGES LIKE CODE

- Store all schema changes in **version control (Git)**.
- Baseline** data BEFORE release via export, snapshot or data copy, (choose base off performance/size)
- Use **branching strategies** (e.g., GitFlow) for database migrations.
- Apply database changes via **automated CI/CD pipelines**.

AUTOMATE DATABASE MIGRATIONS

- Use **enterprise change release tools** to manage schema changes.
- Run **migrations in lower environments first** (Dev → Test → Prod).
- Automate **rollback scripts and data restoration** for failed deployments.

IMPLEMENT CI/CD TESTING ROUTINES

- Write **unit tests** for stored procedures and views.
- Validate **data integrity** before and after schema changes, even if you're only doing so on a subset of full dataset.
- Perform **load and performance testing** before pushing to production.

A vertical bar on the left side of the slide with a gradient from orange at the top to blue at the bottom.

USE FEATURE FLAGS FOR CHANGES

- **Deploy** schema changes **without enabling** them immediately.
- **Enable** new database features **incrementally** via feature toggles.

MONITOR AND OPTIMIZE PERFORMANCE REGULARLY

Automate	Automate indexing and partitioning strategies based on query performance.
Query	Collect query execution metrics to optimize slow queries.
Implement	Implement database anomaly detection using monitoring tools.

SECURE DATA AND ENSURE COMPLIANCE



Automate **database access control** (e.g., Role-Based Access Control).



Mask or encrypt sensitive data in **non-prod environments**.

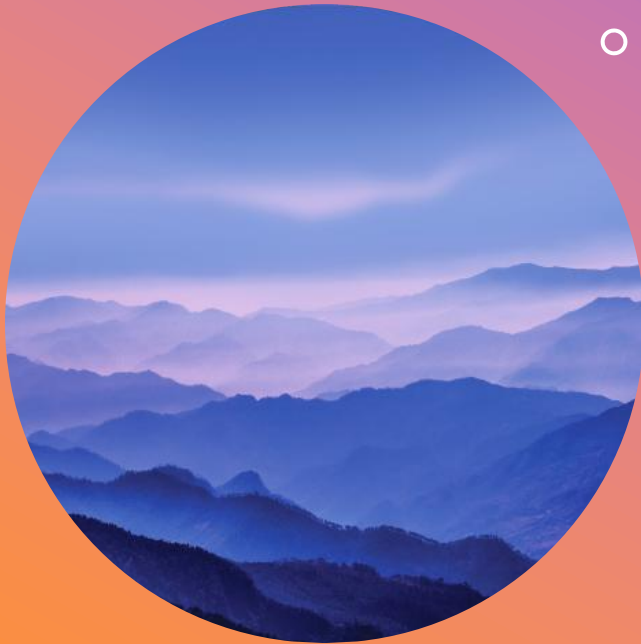


Integrate **audit logging** for all database changes.

BY FOLLOWING...



these best practices and leveraging the right tools, teams can **reduce risk, accelerate development, and improve database reliability** in a DevOps-driven environment.



THANK YOU

Kellyn Gorman
Multiplatform Advocate and Engineer
Redgate

Kellyn.Gorman@red-gate.com

<https://linkedin.com/in/kellyngorman>

<https://dbakevlar.com>



SQLSATURDAY

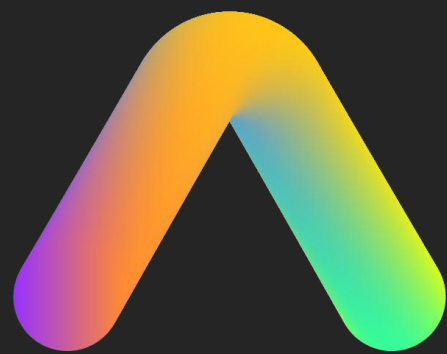
#1103 - Jacksonville, FL May 3rd, 2025

Event Evaluation

Fill out event evaluation card in your bag and visit all sponsors to be entered to win an Xbox Series X – (Must be present to win)

#SQLSatJax





PASS

Data Community Summit

See you in Seattle!

Get \$150 off a 3-day ticket!

Use code: **SQLSATJACKS150**

Register now!

passdatacommunitysummit.com